Contain, Conform, Compare: A Red-Teamed Kit for Evaluating Small Coding Agents on One GPU

Attacked per-job containers, token-level prompt checks, and a pre-registered test of four prompt divergences

Nguyen Minh Thang
VNU University of Science, 334 Nguyen Trai, Thanh Xuan, Hanoi, Vietnam

November 2026

Abstract

A hosted score in the Gemma 4 Developer Agent competition [1] is one noisy number, so development needs a local evaluator that is fast, safe for agent-written code, and matching the hosted scorer's prompts. `drydock-eval` evaluates a small coding agent (Gemma 4 E4B [2]) at about 250 runs per hour on one RTX 5090. *Contain* runs each job in a fresh container without network and passed 200 scripted escape tests; *conform* provides eight checks that compare a local stack's requests with the scorer's configuration, down to rendered tokens; *compare* provides paired statistics that respect repository clusters. AI agents built and attacked the kit; a human owner approved machine use and release. Across seven runner versions no attack escaped the container's namespaces or privileges, yet nine demonstrated high-severity vectors came through what the host shared with it, in designs passing their own escape tests; the last independent verifier found no high or medium issue. Rendered prompts matched the Kaggle template on 396 of 396 requests by token count and 50 of 50 by token id. A pre-registered five-arm experiment (231 tasks, 4,620 jobs) measured four prompt divergences: no E4B solve-rate change was detected (largest estimate -2.1 points, 95% interval -4.4 to $+0.4$).

1 Introduction

The competition scores an agent bundle on hidden GitHub-issue tasks, with Gemma 4 served by vLLM [11]. Third-party analyses put the noise of one hosted score near 1.5 of 58 tasks [3, 4]. Local evaluation must therefore carry development; on a Kaggle kernel with four L4 GPUs our stack finished 35.2 agent runs per hour, serially.

Safety. An agent run executes model-written shell commands and installs model-edited packages; our first local setup ran them with no sandbox.

Fidelity. We call a *rendering trap* a configuration detail that silently changes the token sequence the model receives while runs still finish and score. The *scorer path* is the scorer's configuration reproduced locally. In our first setup the served model name contained "gemma4", so the agent framework (google-adk 1.36.1) sent tool results with role `tool_responses` instead of `tool` [5]. Prompts rebuilt from its traces matched the server's token counts on 106 of 114 calls only when rendered with `tool_responses` (0 of 114 with `tool`). Solve counts hid it: that setup solved 3 and 5 of 24 tasks against 4 and 4 on Kaggle (Fisher p 0.74 and 0.75).

These traps are public: the role rule [5], thought handling in history [6, 7, 8], a newline after thinking text [9], and the scorer's compaction settings [10]. We discovered none of them; we

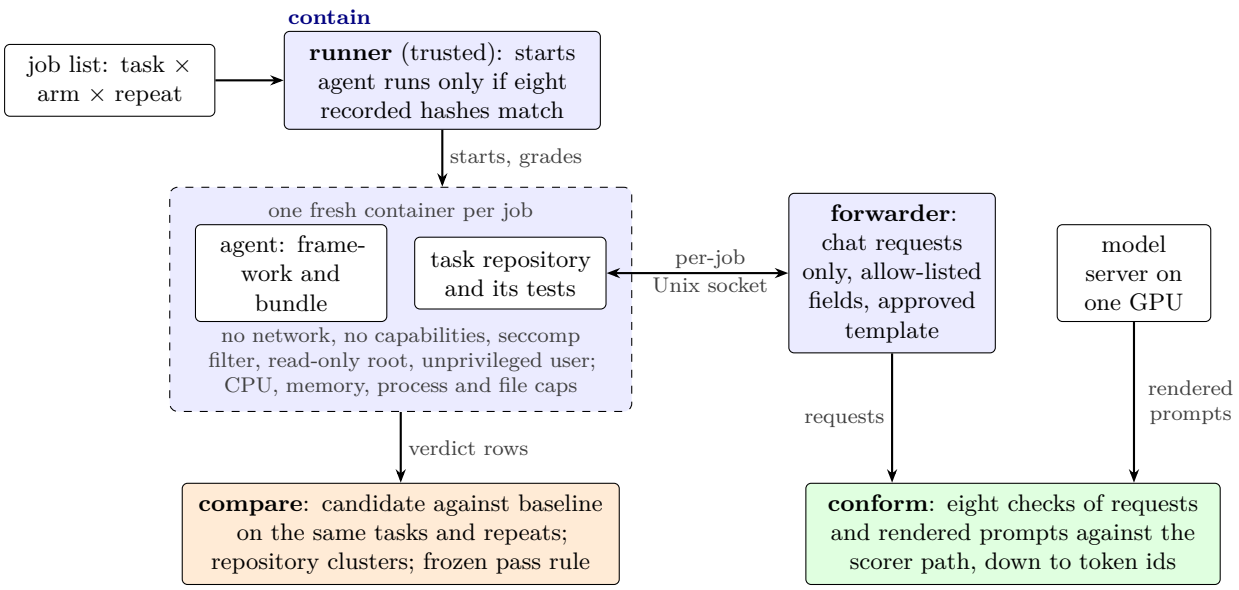


Figure 1: The kit: where each module sits.

contribute checks that detect them (§4) and a measurement of three of them plus a template swap (§6).

2 Scope, threat model and rules

Stack. Gemma 4 E4B, 4-bit quantization-aware checkpoint (Kaggle model version 2); vLLM 0.19.1; google-adk 1.36.1; the competition's harness packages. Tasks come from Berth [12]: 579 of its 653 tasks load in the harness and pass our validation (the reference patch passes, an empty patch fails). Experiments ran on a shared single-GPU workstation (RTX 5090, 32 GB).

Threat model. The adversary is the code the agent writes. It may try to read host files or other jobs, reach the network, exhaust host resources, forge its verdict, or outlive its job. Kernel exploits are out of scope.

Who did the work. AI agents wrote the kit's code, attacks and analyses (builders). Separate AI agents (verifiers) re-implemented or re-ran the work adversarially. A lead AI session froze the protocols; a human owner approved isolation, machine time and release. "Verified" below means this process, not human review.

3 Contain: a runner and the attacks it failed

In the final design each job gets one fresh container: no network, no capabilities, a seccomp filter, a read-only root, an unprivileged user, 2 CPUs, 10 GiB of memory, 4,096 processes, and temporary filesystems capped in bytes and inodes. The only request channel is a per-job Unix socket to a forwarder, which passes only chat-completion requests with allow-listed fields, text-only content and an approved template hash, under rate and size caps. Agent runs start only if eight recorded hashes (image, code, flags, model files) still match.

A *canary* is a scripted attack with a machine-checked outcome.

Passing canaries proved little. No attack escaped the container's namespaces or privileges in any round. Nine demonstrated high-severity vectors came through what the host shared with it:

Table 1: Runner versions.

Runner version	Canaries passed	Security findings of independent verifiers
1. Container per command	52/52	4 high: host-side install ran the agent's `setup.py` (builder-found); symlink leaked a host file; 10 GiB of host disk reserved in 0.005 s; host copied a sparse file in full
2. Hardened, per command	60/60 (57/60 before canary fixes)	4 high, 1 untested: host overwrote a file through a planted link; cleanup followed links; 6 GiB reserved in 0.012 s; 200,000 files in 2.4 s
3. Container per job	98/98	1 high: the model server could fetch URLs named in requests; 2 medium
4–6. Request filter added, tightened	134/134, 159/159, 191/191	medium at most
7. Final	200/200	no high or medium

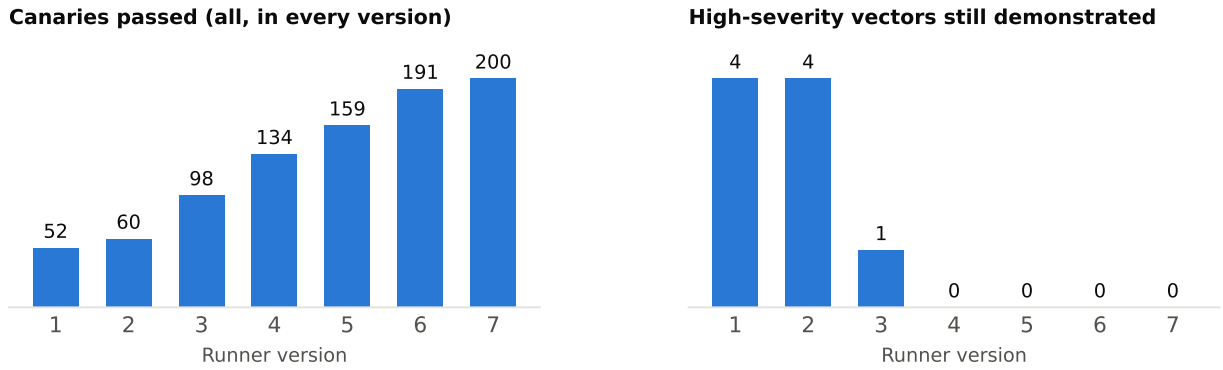


Figure 2: Canaries passed and high-severity vectors found, per runner version.

host-side steps that touched agent-written files, resource limits (a known class [13]) and the model server.

Final version. Controls on 481 tasks (reference patch 481/481, empty patch 0/481) showed no verdict, exit-code or test-summary difference from the previous version. After the runner was killed, the container was gone within 149 s with no further model request. The final runner completed a 24-job list at 251–254 runs per hour with 8 workers.

What remains. Containers share the host kernel; user namespaces are not remapped; the request parser runs in the trusted runner; a job can slow others through the shared model server within its caps. Our claim is conditional: against the stated adversary and with the eight hashes matching, no tested attack succeeded.

4 Conform: eight checks against the scorer path

`conform` reads the requests a local stack sent and, where logged, the prompts the server rendered. Reference values come from the public starter notebook [10], our Kaggle-kernel traces, and the Kaggle model's chat template.

Role. From each google-adk release's source (only 1.36.1 was executed): 1.34.0–1.39.1 and 2.0–2.4 send `tool_responses` when the model string contains "gemma4"; 2.5–2.11 when it matches `gemma-?4`; 2.12–2.14 exempt three OpenAI-compatible providers [5]. The harness packages require `>=1.34.0,<2.0`.

Table 2: The eight checks.

Check	Compares	Mutation runs caught
role	role of tool results	53/53
sampling	temperature, top-p, token and thinking budgets	52/52
tools	declared tool schemas	20/20
shape	message skeleton, system text, field encoding	72/72
template	rendered prompt against the Kaggle template	13/21 by token count, 24/24 by token id
thoughts	history thoughts, under the key the server reads	52/52
newline	newline before the token closing a thought	12/12
compaction	when history is summarized (14,336 tokens [10])	19/19

Thoughts. vLLM 0.19.1 reads an earlier assistant thought only from the field `reasoning`. google-adk 1.36.1 writes `reasoning_content`, which that server drops silently [8]; the competition harness writes both. Removing `reasoning` in a probe lost 55 of 55 thoughts from the rendered prompt.

Template. The check re-renders each request outside vLLM with the Kaggle template and compares token counts with the server's, or token ids when the server logs them, because identical strings can tokenize differently [16].

Evidence. In version 2's 24-task parity run, all 2,566 tool results had role `tool`, and re-rendering reproduced the server's token count on 396 of 396 requests (with `tool_responses`: 0 of 358 multi-turn requests). With the final runner, the checker reproduced the token ids of 243 of 243 returned calls, and a renderer written separately by the verifier matched 50 of 50 scorer-path requests. Prompts rebuilt from early calls of our Kaggle-kernel traces match the recorded counts on 181 of 189 calls with role `tool` and thoughts kept, 1 of 189 with `tool_responses` and 0 of 189 with thoughts dropped: our only Kaggle-side evidence, and count-level.

Mutation suite. 103 mutations (85 drifts, 18 benign controls) in four scripted scenarios against a mock server give 412 runs. The intended check caught 317 of 325 applicable drift runs; the 8 misses are same-length template changes seen through counts only. No check failed in 72 control runs (24 warned).

5 Compare: a decision rule that respects repositories

Tasks from one repository succeed and fail together. Clustered inference for model evaluations is prior art [19]; `compare` packages a small-cluster version with a frozen pass rule and its simulated behaviour.

Candidate and baseline run on the same tasks and repeats. A candidate passes only if (1) the mean gain is at least 2/58 (3.45 points, two leaderboard tasks); (2) one-sided $p \leq 0.025$ under a null-imposed wild cluster bootstrap-t over repositories, with Rademacher weights and 9,999 draws [20, 21]; (3) at least three repositories gain and every leave-one-repository-out estimate is positive; (4) agent time stays within 1.2 times baseline and infrastructure errors do not rise; and (5) no repository loses two task-equivalents or more. Two panels were frozen before any outcome: a small one (72 tasks, 18 repositories) and a large one (200 tasks, 20 repositories).

The 2.5% level holds for exchangeable effects, not for skewed ones, and not if tasks are dropped after unblinding. Independent re-implementations matched the rule code on 280 of 280 clean datasets (first version) and, with rule readings aligned, 193 of 194 (amended). Applied three times, to screen leaders from one separate 48-task set (25, 22, 25 of 96 runs solved; baselines 19, 21, 20);

Table 3: Simulated pass rates (3,000 simulations per cell; 400 for the dropping row).

Simulated scenario	Small panel	Large panel
No effect, exchangeable tasks	0.23–1.90%	0.00–0.50%
No mean effect, candidate breaks 20% of repositories	7.50%	8.43%
No effect, worst tasks dropped after seeing outcomes	31.75%	40.75%
Even gain of 3/58 (power)	≈60%	≈90%
Gain confined to four repositories (power)	15.9%	24.9%

Table 4: Arms; changes in percentage points.

Arm	Difference from the scorer path	Requests changed by the divergence	Change against F [95% interval]; Holm p
F	none: role `tool`, Kaggle template, compaction at 14,336 tokens, thoughts under both keys	—	solve rate 23.8%
R	model name contains "gemma4": tool results get role `tool_responses`	89%	−0.3 [−2.8, +2.2]; p = 1
C	compaction unset: prompts grow until rejected above 24,576 tokens (the run ends; edits are still graded)	21%	+0.2 [−2.0, +2.5]; p = 1
H	history thoughts removed from every request, as when a client omits the key the server reads	90%	−2.1 [−4.4, +0.4]; p = 0.44
T	the Hugging Face E4B template: a thought opener is forced after each tool result	85%	+0.8 [−1.4, +2.9]; p = 1

none passed. Gains: −0.35 points on the small panel (p = 0.60); on the 128 large-panel tasks outside it, −0.39 (p = 0.60), then +4.10 (p = 0.04 > 0.025, uncorrected). Condition (5): first application only.

6 Case study: four prompt divergences on E4B

Five arms share one model server, one withheld agent bundle, google-adk 1.36.1 and all sampling settings.

No newline arm was run: both templates render it identically on this server.

Panel. Removing tasks used by other panels, reserved repositories, slow tasks and duplicates from the 579 leaves 231 tasks in 15 repositories. They run 4 repeats in 5 arms (4,620 jobs); for each task and repeat the arms run back to back in counterbalanced order.

Analysis, frozen before any outcome (protocol sha256 `070c9655d96372d054f430727259de7b25af2ca93c3ec4c9295dfdc37aeb8cf4`). The estimand d_X is the mean over tasks of arm X's solve rate minus arm F's. The primary test is an exact two-sided task-level sign-flip test, Holm-corrected over the four arms at 0.05; only then do we say that X raises or lowers E4B's solve rate *on these tasks*. Intervals (unadjusted) invert that test. A repository-cluster bootstrap interval is secondary (lower simulated power). A frozen checker re-renders every returned request from the job's dump, matches the server's token count and confirms the job's arm; a failing job is re-run. Outcomes stayed blind until a pre-registered cut-off (2026-11-11 06:00 UTC) and were unblinded once.

Power, simulated before data. The family-wise false-claim rate is 1.2–4.2%. At 231 tasks

an even 5-point drop in one arm is detected with probability 82–100% across the simulated settings; the drop or gain detected with 80% power ranges from below 3 to about 5.2 points.

Deviations recorded blind. Independent validation of the final runner finished only after launch. From about an hour in, a second workload held about 9.6 GiB of GPU memory, so timings are under mixed load. The block stopped three times: after 967 jobs at the runner's open-file limit (cause unknown), after 378 more by an external termination signal on the shared machine, and after 3,074 more when the kernel's out-of-memory killer ended the model server (cause undiagnosed). Each resume ran only never-started jobs; the 24 interrupted jobs were re-run once.

Results. All jobs finished before the cut-off. Every arm has a valid row in 924 of 924 slots, and the checker passed all 4,620 analysed jobs. No pre-emption was recorded; 6.0% of runs took over 240 s, above the pre-registered 5% threshold. No difference from F was detected for any arm (Table 4, Figure 3B; smallest Holm-adjusted p 0.44); the secondary repository-cluster intervals all include zero. The largest estimate is H's −2.1 points (unadjusted p 0.11; −1.3 with time-limit rows as missing). In exploratory measures, H runs averaged 137 s against 78 s and ended unsolved at the bundle's five-minute limit in 11.5% of runs against 1.8%; in C, 12.3% of runs ended unsolved on a rejected over-long prompt. These are null results, not evidence of no effect: the simultaneous 98.75% intervals allow a drop of 5.1 points (H) and a gain of 3.5 (T).

7 Related work

SandboxEval [14] and SandboxEscapeBench [15] test sandboxes and models' ability to escape them.

Several paper-track resources overlap this one (described from their public text only): LAB [22], a Docker sandbox with 11 tests and a bundle gate with 16 detectors validated by fault injection; Szeto et al.'s cloud rig with 141 run folders [23]; gemma4-swe-kit [17], a proxy reproducing the scorer's template, call parsing and compaction on other servers; a dataset card reporting count parity on 107 captured vLLM requests [18]; Silent Zeros [24], a bundle checker tested on 22 broken bundles; and ThoughtKeep [25], 1,470 Gemma 4 31B attempts on thought retention. We did not find in those texts, and offer here: a runner attacked by independent verifiers with its canary misses published, token-id comparison backed by a mutation suite, and a trap measurement on E4B with a per-job prompt check.

8 Limitations

- **Solve-level parity is not refuted, not established.** The evidence (Figure 3A) is one 24-task list, one repeat per run; ten runs solved only six distinct tasks.
- **Kaggle-side facts are partly inferred**: the hosted google-adk release, tool-schema equality, and the identity of the hosted template and weights with the public Kaggle model.
- **Environment differences remain.** One RTX 5090 instead of four L4s; 2 CPUs and 10 GiB per job; a bare `pytest` command, used in 30 of 216 commands in our Kaggle traces, is absent locally.
- **The case study** covers one model, one bundle, one google-adk release and one configuration per divergence. Arm T swaps a whole template; in arm H thoughts still reach compaction summaries.

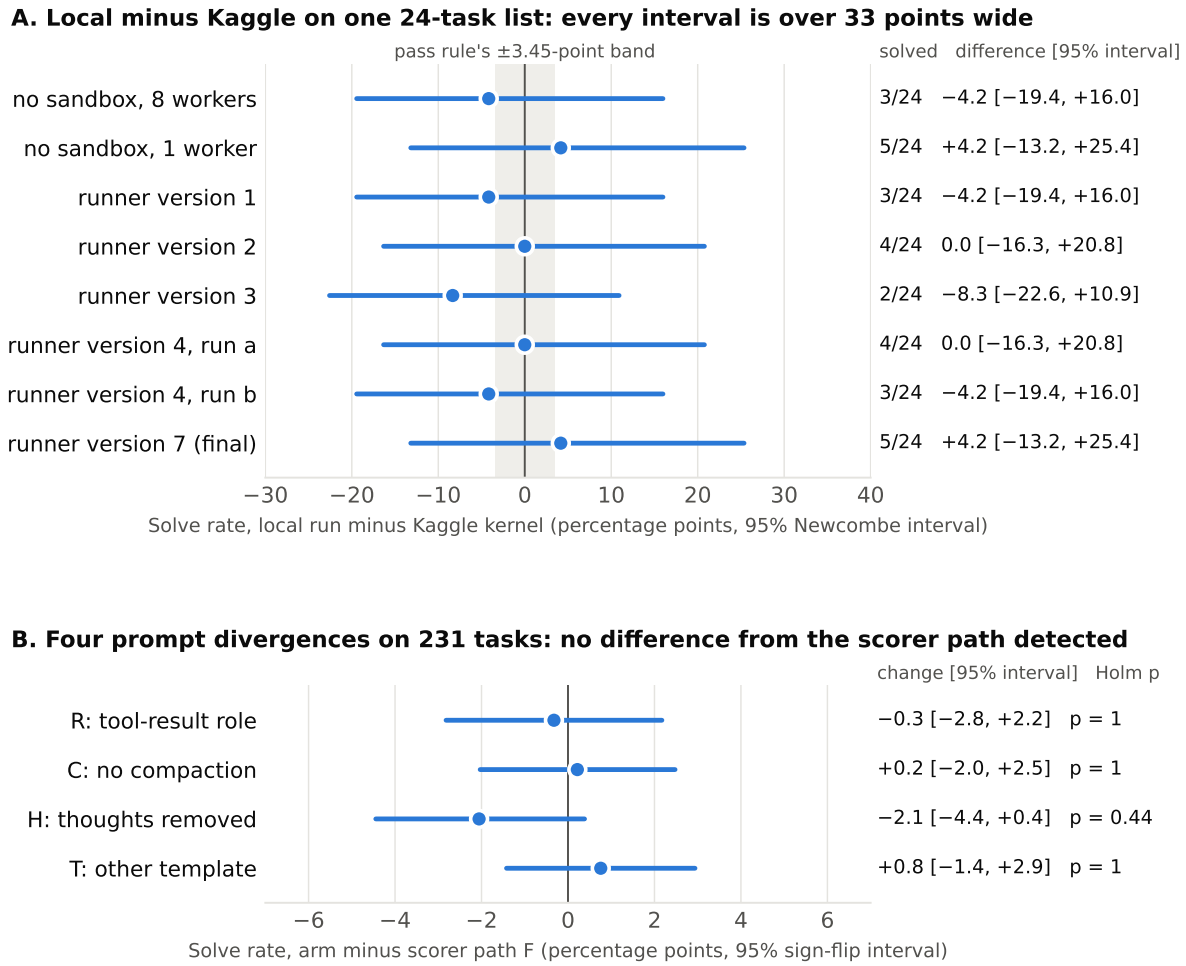


Figure 3: (A) Solve-rate difference, local minus Kaggle (pooled 8/48), for eight local runs of one 24-task list (95% Newcombe intervals, runs treated as independent); shaded: the pass rule's ±3.45 points. (B) Change in solve rate against F on 231 tasks: estimates, 95% intervals and Holm-adjusted p.

9 Availability

`drydock-eval` 0.4.0 (Apache-2.0) has three modules: `conform` (checks, mock server, mutation suite), `contain` (a reference runner tree, its escape tests, a gate helper) and `compare` (pass rule, sign-flip tests, simulations). Its 435 CPU tests pass. The released runner has not been run in a container as released: the 200 of 200 result is for the deployed tree, of which the release is an identifier-substituted copy with two canaries rewritten. Code: https://github.com/MinhThanghus/drydock-eval (tag `v0.4.0`; branch `paper`: supplement, §6 outcome rows). The package also carries copies of the §6 protocol, analysis code and job-list generator, redacted where needed, each with its original's sha256; the agent bundle and request dumps are planned for 2026-12-03.

References

[1] Google DeepMind. Gemma 4 Developer Agent Competition. Kaggle, 2026.

[2] Gemma Team. Gemma 4 Technical Report. arXiv:2607.02770, 2026.

[3] dariushafshar. Gemma 4 Agent: Public Config Draws. Kaggle, 2026.

[4] Mobassir. Too Small to See. Kaggle, 2026.

[5] rladeira. google/adk-python issue #7977. 2026.

[6] Google. Gemma 4 Prompt Formatting. 2026.

[7] dimondev. Hugging Face gemma-4-E2B-it pull request #44. 2026.

[8] delta9000. vLLM issue #38488. 2026.

[9] G. Gerganov. Hugging Face gemma-4-31B-it discussion #135. 2026.

[10] ryanholbrook. Getting Started - Gemma 4 Developer Agent. Kaggle, 2026.

[11] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.

[12] J. Patel. Berth. Kaggle, 2026.

[13] X. Gao et al. Houdini's Escape. CCS 2019.

[14] R. Rabin et al. SandboxEval. arXiv:2504.00018, 2025.

[15] R. Marchand et al. Quantifying Frontier LLM Capabilities for Container Sandbox Escape. ICML 2026.

[16] Agent Lightning Team. No More Retokenization Drift. 2025.

[17] damsolanke. gemma4-swe-kit. GitHub, 2026.

[18] tyfeng1997. gemma4-e4b-swe-sft-32k-competition-template. Hugging Face, 2026.

[19] E. Miller. Adding Error Bars to Evals. arXiv:2411.00640, 2024.

[20] A. Cameron, J. Gelbach, D. Miller. Bootstrap-Based Improvements for Inference with Clustered Errors. REStat, 2008.

[21] J. MacKinnon, M. Webb. Wild Bootstrap Inference for Wildly Different Cluster Sizes. J. Applied Econometrics, 2017.

[22] marcofpereira. Gemma 4 Developer Agent LAB. Kaggle, 2026.

[23] J. Szeto, D. Guo, H. Szeto. Scorer-Faithful Local Evaluation for Small Coding Agents. Kaggle, 2026.

[24] houchiunyo. Gemma 4 agent resource: runs & checker. Kaggle, 2026.

[25] hemantkumarbk. Coding-Agent Thought Retention: Reproduction Data. Kaggle, 2026.

Supplementary material

S1 What this supplement holds

The paper reports the case study of Section 6 and the applications of the pass rule of Section 5 in a few lines each. This supplement gives the full results behind those lines (Sections S2 and S3), recomputes the case study with the released kit (Section S4) and describes the released data (Section S5).

How the numbers were produced. No table cell here was typed by hand. One script reads the result rows of the case study and its frozen analysis report (sha256 `1c13f21eac77efead5520f9c6bfe30318f0523f63e68dfcd245cc3465aacf6df`). From the rows it recounts the solved runs per arm and repeat, the difference of every task, the sum of every repository and the request counts, and it stops if any recount differs from the report. Only then does it write the tables, the numbers used in the text and the data files. The figures are drawn from the released data files alone.

Terms. A *run* is one attempt of the agent at one task, in one arm and one repeat. A run is *solved* if the tests of the task pass on the edits the run left. A *slot* is one combination of task, arm and repeat. *Points* are percentage points of solve rate. A *task-equivalent* is four solved runs, that is, one task solved in all four repeats.

S2 The case study in full

S2.1 Design

Five arms run one withheld agent bundle against one model server (Table 4). Arm F reproduces the scorer path. Each other arm departs from it in one way: in R, tool results carry the role `tool_responses`; in C, history compaction is unset; in H, the thoughts of earlier turns are removed from every request; in T, the Hugging Face template of the model replaces the Kaggle one. The panel has 231 tasks in 15 repositories. Every task runs four repeats in every arm, which gives 4,620 slots, 924 per arm. For each task and repeat the five arms ran back to back in counterbalanced order. The analysis was frozen before any outcome was seen, and outcomes stayed blind until the registered cut-off.

S2.2 Execution record

The first job started on 2026-11-10 06:39 UTC and the last one ended on 2026-11-11 03:56 UTC, before the cut-off (2026-11-11 06:00 UTC). The block ran as four batches and one batch of re-runs (Table S1). Three batches stopped early, for the reasons in the last column, and each following batch started only jobs that had never started. In all, 4,644 jobs were started: the 4,620 analysed runs and 24 interrupted jobs. Of the interrupted jobs, 16 left an infrastructure row; the other 8, all in batch 1, left none and are counted from the list of started jobs. Each interrupted slot was re-run once, so every slot has exactly one analysed run. The released file holds every written row.

276 of the 4,620 runs (6.0%) took more than 240 s of agent time. This is above the 5% share that the protocol set as the sign of a loaded machine: from about an hour into the block a second workload held part of the memory of the GPU (Section 6), so the times are under mixed load. No pre-emption was recorded.

Table S1: Execution record: the block ran as four batches and one batch of re-runs. An interrupted job is an infrastructure row, with or without a written result row; its slot was re-run once.

Batch	Jobs started	Analysed runs	Interrupted, row written	Interrupted, no row	How the batch ended
1	967	959	0	8	runner reached its open-file limit
2	378	370	8	0	external termination signal
3	3,074	3,066	8	0	model server ended (out of memory)
4	201	201	0	0	job list completed
5	24	24	0	0	re-runs of the interrupted jobs completed

S2.3 Outcomes and solve rates

Table S2 splits the runs of each arm by how they ended and Table S3 gives the solve rates, which lie between 21.8% and 24.6%. Two arms end differently from F. In C, 114 runs (12.3%) ended unsolved because a prompt grew past the server's limit and was rejected; no F run ended this way, and 48% of the F runs made at least one history summary. In H, 106 runs (11.5%) were unsolved at the bundle's five-minute limit, against 17 (1.8%) in F.

Table S2: Outcome of every analysed run, by arm (one row per task, arm and repeat).

Arm	Solved	Unsolved, run completed	Five-minute limit	Over-long prompt rejected	Tool-call budget (28)	No submission	Runs
F	220	676	17	0	6	5	924
R	217	677	20	0	9	1	924
C	222	568	17	114	1	2	924
H	201	597	106	1	18	1	924
T	227	668	19	0	4	6	924

Only the runs of the first column are solved; the other columns split the unsolved runs by how the run ended. A run that ended at a limit was still graded on the edits it had made: 5 solved runs lasted the full five minutes (all in arm H), and 86 of the 179 runs unsolved at that limit had left edits that were tested.

Table S3: Solve rate per arm: task-weighted mean with a 95% interval from a repository-clustered (CR1) standard error; solved runs per repeat.

Arm	Solve rate (%)	95% interval	Solved runs	Repeat 1	2	3	4	Runs
F	23.81	[16.7, 30.9]	220	50	59	57	54	924
R	23.48	[17.7, 29.3]	217	53	53	54	57	924
C	24.03	[16.6, 31.5]	222	58	56	55	53	924
H	21.75	[15.9, 27.6]	201	50	49	54	48	924
T	24.57	[18.6, 30.5]	227	59	56	51	61	924

Table S4 shows why a difference between arms is hard to detect on this panel: 135 of the 231 tasks were solved in no run of any arm and 13 in every run, so the other 83 tasks carry every difference.

S2.4 Primary analysis: tasks

For a task, let K be the number of solved repeats in the arm minus the number in F, from -4 to $+4$. The estimate d is the mean of $K/4$ over the 231 tasks. Under the null hypothesis the arm and

Table S4: Tasks by the number of solved repeats (of four), per arm.

Arm	0 solved	1 solved	2 solved	3 solved	4 solved	Tasks
F	149	16	21	18	27	231
R	152	14	21	15	29	231
C	151	15	18	17	30	231
H	155	21	13	14	28	231
T	151	14	13	25	28	231

135 of the 231 tasks were solved in no run of any arm (0 of 20 runs) and 13 in every run; the other 83 tasks carry every difference between arms.

F are exchangeable within a task, so each K is as likely to carry one sign as the other; the test compares $|\sum K|$ with its distribution over all sign assignments, computed exactly. Holm's method adjusts the four p-values. The intervals invert the same test under a constant shift, with 99,999 frozen Monte Carlo sign draws.

No difference from F was detected for any arm (Table S5, Figure S1). In every arm between 172 and 183 of the 231 tasks have the same number of solved repeats as in F. The largest estimate is that of H: 34 tasks have fewer solved repeats than in F and 20 have more, which gives −2.06 points with exact $p = 0.11$ and Holm-adjusted $p = 0.44$. These are null results, not evidence of no effect. The simultaneous 98.75% intervals show what the experiment cannot exclude: a drop of up to 5.1 points (H) and a gain of up to 3.5 points (T).

Table S5: Primary analysis: change in solve rate against F in percentage points, tasks with fewer and with more solved repeats than in F, exact two-sided task-level sign-flip test, Holm adjustment over the four arms, and intervals from inverting the test.

Arm	d	Exact d	Tasks lower; higher	Exact p	Holm p	Monte Carlo p	95% interval	98.75% simultaneous
R	−0.32	−1/308	26; 33	0.8652	1.000	0.8650	[−2.82, +2.17]	[−3.51, +2.85]
C	+0.22	1/462	26; 22	0.9253	1.000	0.9243	[−2.03, +2.48]	[−2.65, +3.09]
H	−2.06	−19/924	34; 20	0.1102	0.441	0.1106	[−4.45, +0.38]	[−5.09, +0.95]
T	+0.76	1/132	25; 30	0.5604	1.000	0.5599	[−1.42, +2.94]	[−2.02, +3.54]

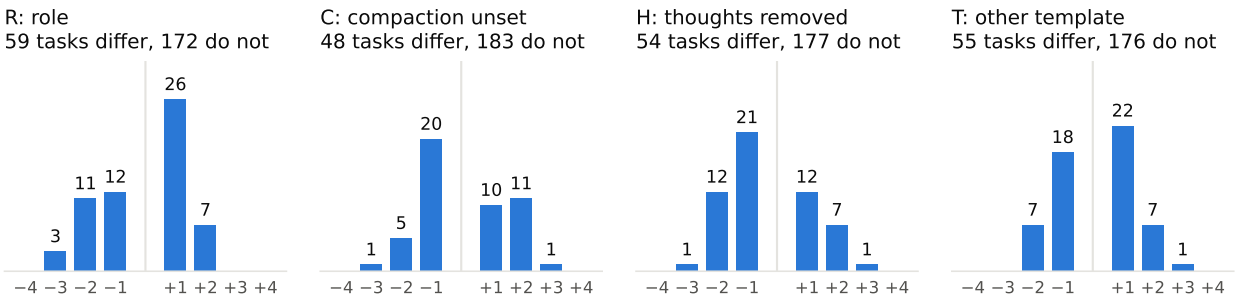


Figure S1: Tasks whose number of solved repeats differs between an arm and F. Bars to the left of the centre line are tasks with fewer solved repeats in the arm.

S2.5 Secondary analysis: repositories

Tasks of one repository tend to succeed and fail together, so the registered secondary analysis takes the 15 repositories as its units (Table S6). Table S7 and Figure S2 show each repository. All four repository-level intervals include zero, and the smallest two-sided p is 0.18 (H). The leave-one-repository-out estimates of H stay negative (-3.01 to -1.44 points), so no single repository produces the estimate of H; those of R, C and T change sign.

Table S6: Secondary analysis at the repository level (15 repositories): exact wild cluster bootstrap-t with Rademacher weights, its inverted 95% interval, the six-point (Webb) variant, a CR1 interval, the repository-equal estimate and the range of leave-one-repository-out estimates. Estimates in percentage points.

Arm	Repositories that differ	Exact p	Webb p	95% interval	CR1 interval	Repository-equal	Leave-one-out range
R	12	0.8188	0.7780	[−3.81, +2.38]	[−2.70, +2.05]	+1.62	[−1.09, +0.36]
C	8	0.8438	0.7788	[−0.88, +2.81]	[−1.22, +1.65]	+4.02	[−0.22, +0.60]
H	12	0.1831	0.1736	[−4.94, +1.56]	[−4.65, +0.53]	−2.98	[−3.01, −1.44]
T	9	0.8047	0.7374	[−1.44, +4.62]	[−2.09, +3.61]	+2.65	[−0.55, +1.66]

Table S7: The 15 repositories of the panel: tasks, runs solved in arm F (of four per task) and, per arm, the summed change against F in task-equivalents (solved runs divided by four).

Repository	Tasks	F solved runs	R	C	H	T
`pypa/packaging`	65	65 of 260	+0.50	−0.50	−2.00	−1.00
`pydantic/pydantic-settings`	48	27 of 192	+1.25	0	+0.75	+2.75
`python-poetry/tomlkit`	35	31 of 140	−1.00	−0.50	−1.50	0
`more-itertools/more-itertools`	23	39 of 92	−1.50	+0.25	−1.75	0
`pallets/click`	22	13 of 88	−0.50	−0.25	−0.50	−0.25
`pallets/werkzeug`	9	3 of 36	+0.25	0	−0.25	+0.25
`mahmoud/boltons`	7	12 of 28	−0.25	0	+0.50	−0.25
`jd/tenacity`	6	9 of 24	−0.25	0	−0.25	−0.50
`pytest-dev/pluggy`	6	3 of 24	+0.50	+1.00	+0.75	0
`python-humanize/humanize`	3	11 of 12	−0.25	+0.25	+0.25	+0.25
`hynek/structlog`	2	2 of 8	+0.25	−0.25	−0.50	+0.25
`python-attrs/attrs`	2	0 of 8	0	0	0	0
`jazzband/prettytable`	1	1 of 4	+0.25	+0.50	−0.25	+0.25
`marshmallow-code/marshmallow`	1	0 of 4	0	0	0	0
`python-attrs/cattrs`	1	4 of 4	0	0	0	0
All	231	220 of 924	−0.75	+0.50	−4.75	+1.75

S2.6 Sensitivity analyses

Three sensitivity analyses were registered with the primary one (Table S8). The first treats the runs that were unsolved at the five-minute limit as missing, in both arms; the estimate of H is then -1.25 points ($p = 0.34$) instead of -2.06. The other two score as unsolved every slot whose first job was interrupted, in the arm or in F. No sensitivity analysis gives a p-value below 0.05.

S2.7 Exposure and the prompt check

A divergence can only matter in the requests it changes. The frozen checker re-rendered all 75,606 returned requests from the request dumps of the jobs. For each of the 4,620 analysed runs it

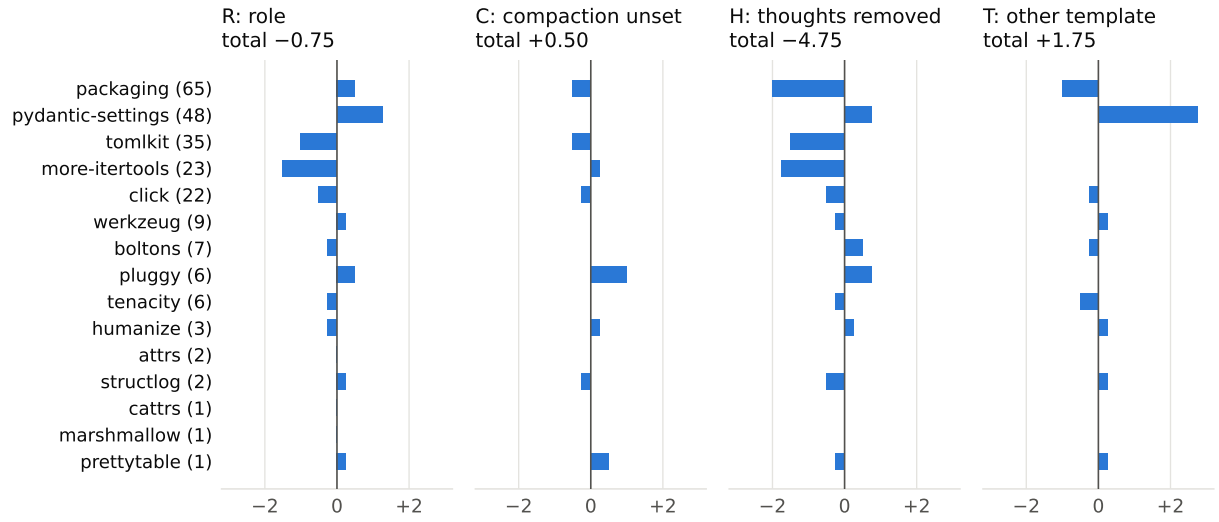


Figure S2: Change against F in each repository, per arm. Repositories are ordered by their number of tasks.

Table S8: Pre-registered sensitivity analyses of the primary estimate (percentage points; exact two-sided sign-flip p, unadjusted).

Arm	Analysis	Tasks	Runs affected	d	Exact p
R	five-minute-limit runs of both arms treated as missing	229	37	−0.29	0.8652
	slots of the arm with an infrastructure row scored unsolved	231	5	−0.43	0.8011
	slots of F with an infrastructure row scored unsolved	231	3	−0.22	0.9321
C	five-minute-limit runs of both arms treated as missing	229	34	+0.22	0.9253
	slots of the arm with an infrastructure row scored unsolved	231	5	+0.11	1.0000
	slots of F with an infrastructure row scored unsolved	231	3	+0.32	0.8506
H	five-minute-limit runs of both arms treated as missing	226	123	−1.25	0.3414
	slots of the arm with an infrastructure row scored unsolved	231	8	−2.16	0.0980
	slots of F with an infrastructure row scored unsolved	231	3	−1.95	0.1270
T	five-minute-limit runs of both arms treated as missing	229	36	+0.76	0.5604
	slots of the arm with an infrastructure row scored unsolved	231	3	+0.76	0.5604
	slots of F with an infrastructure row scored unsolved	231	3	+0.87	0.4947

confirmed that the prompts carry the divergence of the run's arm and no other difference; 0 runs failed the check. The divergence changed 85 to 90% of the requests in R, H and T, and 21% in C (Table S9).

S2.8 Exploratory run measures

Table S10 compares thirteen measures of a run between each arm and F. These comparisons were not part of the registered tests and are not adjusted for multiplicity. H runs are longer: 137 s of agent time on average against 78 s in F (medians 106 and 62 s; Figure S3), with 14,397 completion tokens per run against 7,690.

Table S9: Exposure: requests re-rendered by the frozen checker, and those whose rendered prompt the arm's divergence changed.

Arm	Runs	Requests checked	Changed by the divergence	Share (%)	Failed calls recorded
F	924	15,043	0	0.0	17
R	924	15,290	13,616	89.1	20
C	924	13,036	2,709	20.8	131
H	924	17,450	15,684	89.9	112
T	924	14,787	12,629	85.4	19

No run has more than one failed call. Of the 299 failed calls, 178 are in runs unsolved at the five-minute limit, 115 in runs that ended on a rejected over-long prompt and 6 in solved runs.

Table S10: Exploratory run measures: mean in arm F, and the paired difference of each arm against F; below it, its 95% interval (CR1, repository clusters). No multiplicity adjustment.

Measure	F mean	R − F	C − F	H − F	T − F
history summaries per run	0.69	+0.13	−0.69	−0.30	−0.02
		[+0.04, +0.22]	[−0.86, −0.53]	[−0.47, −0.12]	[−0.10, +0.06]
runs with a summary (share)	0.482	+0.038	−0.482	−0.161	−0.014
		[−0.024, +0.099]	[−0.572, −0.391]	[−0.249, −0.073]	[−0.059, +0.030]
largest prompt (tokens)	12,721	+220	+2,160	−1,499	−95
		[+111, +329]	[+1,666, +2,655]	[−1,903, −1,095]	[−284, +95]
over-long prompt rejected (share)	0.000	+0.000	+0.123	+0.001	+0.000
		[+0.000, +0.000]	[+0.075, +0.172]	[−0.001, +0.003]	[+0.000, +0.000]
five-minute limit (share)	0.018	+0.003	+0.000	+0.096	+0.002
		[−0.005, +0.012]	[−0.005, +0.005]	[+0.073, +0.119]	[−0.002, +0.006]
tool-call budget reached (share)	0.006	+0.003	−0.005	+0.013	−0.002
		[−0.004, +0.011]	[−0.012, +0.001]	[−0.002, +0.028]	[−0.006, +0.002]
no submission (share)	0.005	−0.004	−0.003	−0.004	+0.001
		[−0.015, +0.006]	[−0.013, +0.007]	[−0.015, +0.006]	[−0.010, +0.012]
agent time (s)	77.8	+3.9	−11.2	+58.9	−1.2
		[−0.1, +7.9]	[−14.7, −7.8]	[+49.4, +68.3]	[−6.5, +4.0]
model calls	16.28	+0.27	−2.17	+2.60	−0.28
		[−0.25, +0.79]	[−2.51, −1.83]	[+1.31, +3.90]	[−1.05, +0.50]
failed model calls	0.018	+0.003	+0.123	+0.103	+0.002
		[−0.005, +0.012]	[+0.077, +0.169]	[+0.079, +0.127]	[−0.002, +0.006]
tool calls	12.87	+0.19	−1.23	+2.52	−0.23
		[−0.31, +0.70]	[−1.42, −1.05]	[+1.64, +3.41]	[−0.92, +0.45]
prompt tokens, summed	136,123	+3,585	+9,332	+12,785	−3,332
		[−1,493, +8,662]	[+4,824, +13,841]	[+443, +25,127]	[−11,389, +4,725]
completion tokens	7,690	+440	−1,253	+6,707	−245
		[−18, +899]	[−1,635, −871]	[+5,645, +7,768]	[−796, +305]

S3 Applications of the pass rule

The pass rule of Section 5 was applied three times to a real candidate: a changed version of the withheld agent bundle, tested against the unchanged baseline on the same tasks and repeats. Each application ran once. Its rule, tasks and analysis script were frozen before its first job, and the outcomes stayed unread until the run had ended. Table S11 gives every quantity that the rule uses, and Figure S4 the gain of every repository.

How the candidates were chosen. Each candidate was the leader of a screen on one separate 48-task set, with two repeats per task: 576 runs over the baseline and five variants for the first

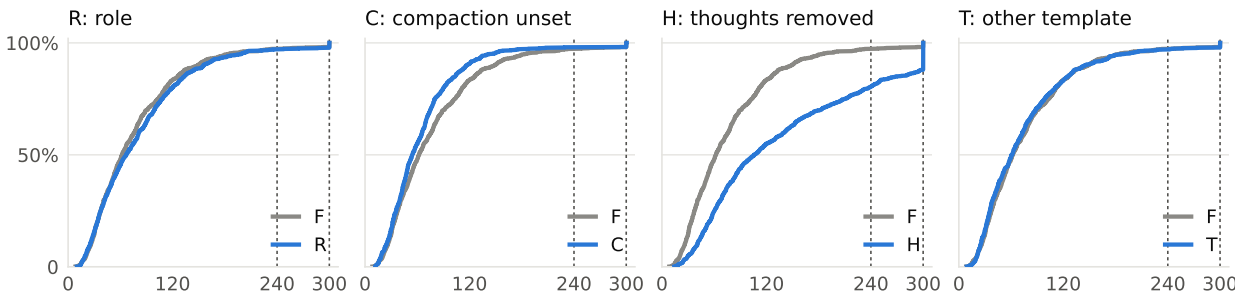


Figure S3: Agent time per run: each arm (blue) against F (grey).

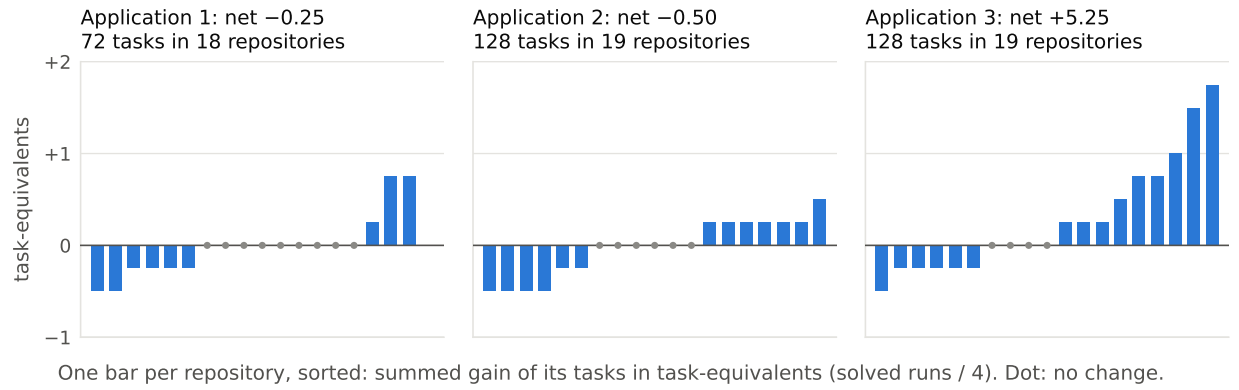


Figure S4: Gain of each repository in the three applications of the pass rule (candidate minus baseline). Repositories are sorted by their total within each application.

candidate, and 288 runs over the baseline and two variants for each of the other two. Each leader solved more runs than the baseline in its screen (25, 22 and 25 of 96, against 19, 21 and 20). A screen selects a candidate and tests nothing, because the leader of several variants is expected to look better in its screen than it is. No task of the screens is among the tasks of an application. Two small pilots came before the screens of the later candidates. The variants of the second candidate were first tried on 15 tasks of the case-study panel, which shares no task with an application. Two predecessors of the third candidate were tried on 16 tasks of the small panel, that is, on tasks of the first application and on none of the second and third.

Tasks and conditions. The first application used the small panel (72 tasks) and all five conditions of Section 5. The second used the 128 tasks of the large panel that are outside the small one. The third used the same 128 tasks as the second, and both arms ran again. Its registration stated three things before the run: that this was the second use of that task set, that the candidate had run on none of these tasks, and that the candidate was built after the result of the second application was known. It also named two mechanical reads of earlier runs that went into a repair of the candidate and that may include runs on these tasks: a scan of the agent's commands in the 1,024 runs of the second application, and the output of one command in six earlier runs. Neither read used whether a task was solved. The candidate had been submitted to the competition once before the run; its hosted score had not arrived when the outcomes were opened, and it is not an input of the rule. The second and third applications registered conditions (1) to (4) only.

Table S11: Every application of the pass rule of Section 5 to a real candidate, from the frozen gate reports. Baseline and candidate are two versions of the withheld agent bundle.

	Application 1	Application 2	Application 3
Task set	small panel	large panel minus small	as application 2
Tasks; repositories; repeats	72; 18; 4	128; 19; 4	128; 19; 4
Runs per arm (all with a result row)	288	512	512
Solved runs: baseline, candidate	63, 62	103, 101	97, 118
Mean gain (points; exact fraction)	−0.35 (−1/288)	−0.39 (−1/256)	+4.10 (21/512)
(1) gain at least 2/58	not met	not met	met
Bootstrap-t; one-sided p (9,999 draws)	−0.175; 0.6034	−0.366; 0.6034	+1.954; 0.0405
One-sided p by exact enumeration	0.6113	0.6067	0.0360
(2) $p \leq 0.025$	not met	not met	not met
Repositories gaining; losing	3; 6	7; 6	9; 6
(3a) at least three repositories gain	met	met	met
Lowest leave-one-repository-out gain (points)	−1.47	−0.85	+2.94
(3b) every leave-one-out gain positive	not met	not met	met
Agent time, candidate / baseline (s)	68.4 / 66.6	89.3 / 75.6	79.4 / 72.0
(4a) ratio at most 1.2	1.026: met	1.182: met	1.103: met
Infrastructure rows: baseline, candidate	0, 0	0, 0	0, 0
(4b) infrastructure errors do not rise	met	met	met
Worst repository total (task-equivalents)	−0.50	−0.50	−0.50
(5) no repository loses two or more	met	not registered	not registered
Decision of the frozen rule	**FAIL**	**FAIL**	**FAIL**

Results. The rule asks for a mean gain of at least 3.45 points with a one-sided p of at most 0.025, and for further conditions on repositories, time and infrastructure errors. No candidate passed. The first two estimates are below zero: −0.35 points (62 against 63 solved runs of 288; $p = 0.60$) and −0.39 points (101 against 103 of 512; $p = 0.60$). The third candidate solved 21 runs more than the baseline (118 against 97 of 512), a gain of +4.10 points. This gain is above the required size, and conditions (3) and (4) are met: 9 of the 19 repositories gain, 6 lose, and the lowest leave-one-repository-out gain is +2.94 points. But the one-sided p is 0.04, above 0.025, so condition (2) is not met and the decision is FAIL.

How to read the decisions. The verdict of a failed application is "not confirmed". It is not evidence that the candidate has no effect: the registration of the third application estimated, before the run, that a run of this size detects a true gain of the required size about half of the time. Nor do we call the third candidate an improvement. Its p is not corrected for the three applications, and it is also above 0.0083, the level divided by three. Its gain is uneven: the three repositories with the largest totals hold 27 of the 128 tasks and gain +4.25 task-equivalents, while the net gain over all repositories is +5.25 (Figure S4). A gain concentrated in a few repositories is the case in which the rule has little power (Table 3), and it is also what a few repositories can produce by chance. The first two leaders did not keep their screen lead on new tasks; this is consistent with the selection effect that the rule exists to catch.

S4 Recomputing the case study with the released kit

The released table `e8_compare_table.csv` is in the input format of the `compare` module of the kit. From a clone of the repository, two commands recompute the analysis:

```
pip install "./drydock-eval[compare]"
drydock-eval compare signflip e8_compare_table.csv --reference F --repeats 4
```

We ran this command from the released source tree with the script `supp/reproduce.py`, which stops on any mismatch. For all four arms the command returned the registered estimate, the exact p as the same fraction, the Holm-adjusted p and every repository-level quantity of Table S6.

Two quantities depend on Monte Carlo draws: the inverted intervals, and the Monte Carlo p that is reported next to the exact one. The draws are frozen (seed 20261109), but which draw meets which task depends on the order of the tasks. The registered analysis used the order of the panel file; the command sorts the task identifiers. With the tasks in panel order, the routine of the kit returns the registered values to the last digit (Table S12). With the order of the command, the ends of the 95% intervals move by at most 0.13 points and those of the simultaneous intervals by at most 0.02 points. The largest move is the upper end of the interval of H, +0.25 instead of +0.38 points. The p-value of the test is a step function of the shift, and across that stretch it stays between 0.0501 and 0.0509, next to the 0.05 it is compared with. Both intervals include zero. The paper reports the registered intervals.

Table S12: Reproduction with the released kit: the two quantities that depend on Monte Carlo draws. 95% intervals of the change against F in percentage points, and the Monte Carlo p that accompanies the exact p.

	95% interval			Monte Carlo p		
Arm	Registered	Kit, panel order	Kit command	Registered	Kit, panel order	Kit command
R	[−2.82, +2.17]	[−2.82, +2.17]	[−2.80, +2.16]	0.8650	0.8650	0.8642
C	[−2.03, +2.48]	[−2.03, +2.48]	[−2.04, +2.48]	0.9243	0.9243	0.9268
H	[−4.45, +0.38]	[−4.45, +0.38]	[−4.45, +0.25]	0.1106	0.1106	0.1095
T	[−1.42, +2.94]	[−1.42, +2.94]	[−1.42, +2.94]	0.5599	0.5599	0.5624

S5 Released data and code

Table S13 lists the data files and Table S14 the columns of the run file.

What the files hold. Outcome rows and analysis results only. They hold no prompt, request, patch, test output, error text or machine detail: the script copies a fixed list of fields by name and nothing else.

What is withheld. The agent bundle, its prompts and the request dumps of the jobs stay private for now; their release is planned for 2026-12-03 (Section 9). For that reason the two arms of each pass-rule application are named baseline and candidate.

Where. The kit is at `https://github.com/MinhThanghus/drydock-eval` (tag `v0.4.0`, Apache-2.0). The branch `paper` of the same repository holds the sources of the paper and of this supplement, the scripts that write its tables and figures, and the data files with their checksums.

Table S13: Data files released with the paper.

File	Records	Content
`e8_runs.csv`	4,636	one line per result row of the case study: arm, task, repository, repeat, outcome, times, calls, tokens, prompt check
`e8_compare_table.csv`	4,636	the same rows in the input format of `drydock-eval compare` (seven columns)
`e8_tasks.csv`	231	per task: repository and solved repeats (0 to 4) in each arm
`e8_repositories.csv`	15	per repository: tasks, solved runs per arm, summed change against F
`e8_summary.json`	–	the analysis results behind Tables S3 to S10, copied field by field from the frozen report
`pass_rule_applications.json`	3	the applications of the pass rule (Table S11)

Table S14: Columns of `e8_runs.csv`. The rows of interrupted jobs hold the first ten columns and the two times of day only.

Column	Content
`batch`	1 to 5: the batch of the block in which the job ran; batch 5 holds the re-runs
`stage`	1, 2 or 3: the nested stage of the job list that holds the task (118, 37 and 76 tasks)
`arm`	F, R, C, H or T
`task`	task identifier in the public task collection the panel was drawn from
`repository`	repository of the task (owner/name); the unit of clustering
`repeat`	1 to 4
`attempt`	1; 2 for the re-run of an interrupted job
`row_class`	agent: an analysed run. infrastructure: an interrupted job, not analysed
`outcome_id`	A00 solved; A01 unsolved, run completed; A02 tool-call budget reached; A05 unsolved at the five-minute limit; A06 no submission; A18 unsolved, over-long prompt rejected; I09 interrupted job
`resolved`	1 if the run is solved (the tests of the task pass on the edits the run left), else 0
`t_agent_s`	seconds of the agent phase of the job
`t_setup_s`	seconds of the setup phase, before the agent starts
`t_verify_s`	seconds of the grading phase; empty if no test was run
`model_calls`	chat requests of the run that returned a response
`model_calls_failed`	chat requests of the run that raised an error instead
`tool_calls`	tool calls made by the agent
`tokens_in`	prompt tokens, summed over the requests of the run
`tokens_out`	completion tokens, summed over the requests of the run
`max_prompt_tokens`	largest prompt of the run, in tokens
`summaries`	history summaries (compactions) made during the run
`patch_chars`	characters of the diff the run left; 0 if none
`test_exit`	exit status of the test command; 0 = tests passed; −1 = no test was run
`start_utc`	start of the job (UTC)
`end_utc`	end of the job (UTC)
`requests_checked`	returned requests of the run that the frozen checker re-rendered
`requests_changed`	of them, requests whose prompt the reference rule of arm F would render differently (the exposure count; 0 in F)
`check_passed`	1 if the run passed the prompt check